

PROVISIONAL PATENT APPLICATION

Method and System for Generating, Scoring, Signing, Anchoring, and Verifying Verifiable Entropy Objects

Inventor: [Full Legal Name]

Date: June 25, 2026

Attorney Docket No.: [To be assigned]

CROSS-REFERENCE TO RELATED APPLICATIONS

[0001] This application is related to U.S. Patent Application [Serial No. — "Method and System for Gaming Random Number Generation"], which describes a Merkle-tree-based random number generation and verification system with blockchain anchoring.

[0002] This application is also related to U.S. Patent Application [Serial No. — VDF Integration], which describes a Verifiable Delay Function pipeline integrated with the Merkle batch system for anti-preview entropy generation.

[0003] The present invention extends the prior applications by introducing a structured, self-describing, cryptographically verifiable entropy object format; a multi-dimensional entropy quality scoring system; a deterministic canonicalization and signing protocol; and a multi-level verification framework.

FIELD OF THE INVENTION

[0004] The present invention relates to systems and methods for generating, representing, evaluating, cryptographically signing, blockchain-anchoring, and independently verifying entropy as a structured digital object. More specifically, the invention relates to a Verifiable Entropy Object (VEO) standard, an Entropy Confidence Score (ECS) system, a canonical signing protocol for entropy attestation, and a verification framework that distinguishes structural validity from cryptographic verification from blockchain-anchored verification.

BACKGROUND OF THE INVENTION

The Problem with Traditional Random Number Generation

[0005] Conventional random number generators, including hardware RNGs, pseudo-random number generators, and cryptographic random number generators, produce numerical values as their output. Once generated, these values carry no metadata about their origin, generation process, source quality, or integrity guarantees. The generation process is discarded, and the consumer must trust the generator implicitly.

[0006] This trust model is increasingly insufficient for modern applications where random or non-deterministic decisions have significant consequences:

[0007] **AI Agent Systems.** Autonomous AI agents make decisions involving randomness, including tool selection, response sampling, task routing, and resource allocation. These decisions cannot be audited for fairness because the randomness carries no provenance.

[0008] **Regulated Gaming.** Casino and lottery systems are required by regulators to prove that random outcomes were generated fairly. Current systems rely on audit logs produced by the same system being audited, creating a circular trust dependency.

[0009] **Decentralized Systems.** Blockchain protocols, fair-launch mechanisms, and multi-party computations require randomness that is verifiably uncontrollable by any single party, including the generator.

[0010] **Scientific Simulation.** Monte Carlo simulations, stochastic modeling, and experimental protocols require reproducible and auditable entropy sources to enable peer verification of published results.

Limitations of Existing Approaches

[0011] **Verifiable Random Functions (VRFs).** VRFs prove that a specific output was derived from a specific input using a specific key. However, VRFs prove derivation, not entropy quality. They typically use a single entropy source, do not quantify confidence, do not carry source provenance metadata, require on-chain callback mechanisms, and do not support offline verification.

[0012] **Quantum Random Number Generators (QRNGs).** QRNGs produce high-quality entropy from quantum mechanical processes. However, QRNG services return raw numerical values without structured provenance, quality scoring, or independent verification capability. The consumer must trust the QRNG provider.

[0013] **Public Randomness Beacons.** Systems such as drand produce publicly verifiable random values. However, these are single-source systems that do not aggregate entropy from multiple independent sources, do not score entropy quality, and do not produce self-describing verifiable objects.

[0014] There exists a need for a system that transforms entropy from a disposable numerical output into a structured, self-describing, multi-source, quality-scored, cryptographically signed, optionally blockchain-anchored, and independently verifiable digital object.

SUMMARY OF THE INVENTION

[0015] The present invention provides:

[0016] First, a **Verifiable Entropy Object (VEO) format** — a structured digital object that encapsulates entropy payload, source provenance records, aggregation metadata, quality scoring, cryptographic proof, optional blockchain anchoring, and lineage references.

[0017] Second, an **Entropy Confidence Score (ECS) system** — a multi-dimensional, weighted quantitative assessment of entropy quality, producing a score from 0 to 1000 across six independently computed dimensions with defined grade boundaries and fallback penalty rules.

[0018] Third, a **canonical signing protocol** — a deterministic method for transforming a VEO into a signing payload through deep key sorting, field inclusion and exclusion rules, and stable serialization, enabling cryptographic signatures that any party can independently verify through address recovery.

[0019] Fourth, a **multi-level verification framework** — a structured verification output that distinguishes among five levels of trust: structural validity, cryptographic verification, blockchain-anchored verification, policy failure, and invalidity.

[0020] Fifth, a **consumer policy system** — predefined and custom policy presets that specify minimum entropy quality, source count, and anchoring requirements, with honest failure when requirements cannot be met.

[0021] Sixth, an **entropy provenance graph** — a lineage mechanism where each entropy object may reference parent objects, creating a directed acyclic graph of entropy derivation that enables full audit trails.

DETAILED DESCRIPTION OF THE INVENTION

1. Verifiable Entropy Object (VEO)

1.1 Definition

[0022] A Verifiable Entropy Object (VEO) is defined as a cryptographically verifiable representation of uncertainty, including entropy payload, source provenance, confidence metrics, verification metadata, and trust assertions.

1.2 Object Classes

[0023] The invention defines a classification system for entropy objects based on their trust characteristics:

Class	Name	Description
VEO-1A	Raw Source	Entropy from a single source (randomness beacon, block hash, QRNG)
VEO-1B	Composite	Entropy aggregated from two or more independent sources
VEO-1C	Anchored	Composite entropy anchored to an immutable ledger
VEO-1D	Decision	Entropy linked to a specific decision or action

1.3 Required Fields

[0024] Each VEO contains the following required fields: a protocol standard identifier; a protocol version; a unique object identifier; an object class designation; a hexadecimal-encoded entropy payload; a SHA-256 hash of the entropy payload; an ISO 8601 generation timestamp; a provider identifier; an array of entropy source records; a cryptographic proof package; and an Entropy Confidence Score object.

1.4 Entropy Source Records

[0025] Each source record contains a source identifier, a source type classification, a source reference, a fetch timestamp, an individual source entropy hash, and an optional source-provided signature.

1.5 Multi-Source Aggregation

[0026] When multiple sources are used, the system aggregates entropy deterministically. The entropy values from each source are normalized to lowercase, concatenated with a delimiter character, and hashed using SHA-256. The aggregation record preserves the method identifier, input ordering, and resulting hash, enabling any verifier to reproduce the aggregation from the individual source records.

2. Entropy Confidence Score (ECS)

2.1 Purpose

[0027] The ECS quantifies entropy quality as a single integer score from 0 to 1000, enabling consumers to programmatically evaluate whether an entropy object meets their requirements without inspecting individual sources.

2.2 Dimensions and Weights

[0028] The ECS is computed from six independently calculated dimensions, each producing a sub-score from 0 to 1000:

Dimension	Weight	Measurement
Freshness	20%	Time elapsed since generation; decays linearly
Diversity	15%	Ratio of unique live source identifiers to target count
Independence	20%	Ratio of unique source type categories to target count
Manipulation Resistance	20%	Difficulty of controlling aggregated output
Verification Success	15%	Rate of successful source verification
Availability	10%	Source availability at generation time

[0029] The composite score is computed as the weighted sum of the dimension sub-scores, clamped to the range 0–1000.

2.3 Grade Boundaries

Score Range	Grade
900–1000	AAA
800–899	AA
700–799	A
600–699	B
500–599	C
0–499	LOW

2.4 Fallback Penalty System

[0030] When a source adapter fails to retrieve live external entropy and falls back to local cryptographic randomness, defined penalties are applied. For one fallback source, the verification success dimension is reduced by 100 points and manipulation resistance is reduced by 50 points. For two fallback sources, verification success is reduced by 250, manipulation resistance by 150, and diversity by 150. When all sources fall back, all penalties are applied, the total score is capped at 650, and the grade cannot exceed B.

[0031] The confidence metadata explicitly reports the fallback count, live source count, and an overall source status indicator, ensuring the score never misrepresents entropy quality.

3. Canonical Signing Protocol

3.1 Canonicalization Rules

[0032] Before signing, a VEO is transformed into a deterministic canonical form using the following rules: (a) all object keys at every nesting level are sorted alphabetically; (b) undefined values are removed; (c) null values are preserved

explicitly; (d) stable JSON serialization is applied to the sorted structure; (e) the same input must always produce the same canonical byte string.

3.2 Field Inclusion and Exclusion

[0033] The canonical signing payload includes the following VEO fields: standard, version, object identifier, object class, entropy, entropy hash, issuance timestamp, expiration, provider, sources, aggregation, confidence, lineage, and policy. The anchor field is always set to null in the canonical payload because the signature is computed before blockchain anchoring occurs.

[0034] All fields within the proof object are excluded because the signature cannot include itself. All fields within the anchor object are excluded because transaction hash, block number, and timestamp are only known after anchoring.

3.3 Signing Process

[0035] The canonical payload is hashed using SHA-256. The hash is signed using EIP-191 personal message signing with the provider's secp256k1 private key. The provider's Ethereum address is recoverable from the signature, enabling any party to verify the signature without access to the private key.

4. Multi-Level Verification Framework

4.1 Verification Levels

[0036] The system defines five verification levels representing increasing degrees of trust:

Level	Requirements
Structurally Valid Unsigned	Schema valid; entropy hash matches; no signature; no anchor
Cryptographically Verified	Schema valid; hash matches; provider signature valid
Anchored Verified	Schema valid; hash matches; signature valid if present; anchor verified on-chain
Policy Failed	Object may be valid but does not meet consumer policy
Invalid	One or more verification checks failed

4.2 Verification Checks

[0037] The verification output includes individual check results for: schema validity, hash integrity, signature verification (true, false, or null), source record validity, confidence score range, blockchain anchor verification (true, false, or null), and policy compliance.

4.3 Signature Verification

[0038] To verify a provider signature, the system reconstructs the canonical signing payload from the VEO, computes its SHA-256 hash, recovers the signer's Ethereum address from the EIP-191 signature, and compares the recovered address to the claimed provider address. A match yields a positive verification; a mismatch yields a negative verification; the absence of a signature yields a null result.

4.4 Anchor Verification

[0039] To verify a blockchain anchor, the system queries the on-chain smart contract using the batch identifier from the anchor metadata, retrieves the stored Merkle

root, and compares it to the Merkle root recorded in the VEO anchor metadata. A match confirms the anchor is valid.

5. Consumer Policy System

[0040] The system provides predefined policy presets that consumers may invoke by name. Each preset specifies a minimum ECS score, minimum source count, and whether blockchain anchoring is required. When the system cannot satisfy a requested policy, it returns an explicit error rather than returning a degraded entropy object at a higher trust level. This prevents false trust assertions.

6. Entropy Provenance Graph

[0041] Each VEO may reference parent entropy objects by their object identifiers, creating a directed acyclic graph of entropy lineage. A lineage hash is computed over the sorted parent object identifiers, enabling verification that the claimed lineage has not been altered. This enables audit trails that trace a final entropy value back through every source that contributed to it, supports replay by reconstructing a VEO from the same source inputs, and provides cryptographic evidence for dispute resolution.

7. Blockchain Anchoring

[0042] For applications requiring the highest trust level, the entropy hash is anchored to a blockchain smart contract. The system computes the entropy hash, uses it as a Merkle leaf, submits the Merkle root to the on-chain anchoring contract, waits for block confirmations, reads back the stored root to confirm it matches, and records the transaction hash, block number, batch identifier, and on-chain timestamp in the VEO anchor metadata. The object class is upgraded to VEO-1C only after successful anchoring and readback verification.

[0043] Any party can independently verify the anchor by querying the smart contract directly using the batch identifier and comparing the on-chain root to the root recorded in the VEO. No trust in the VEO provider is required.

CLAIMS

What is claimed is:

1. A computer-implemented method for generating a verifiable entropy object, comprising: (a) obtaining entropy from a plurality of independent entropy sources; (b) computing an individual entropy hash for each source; (c) aggregating the entropy from the plurality of sources using a deterministic aggregation function; (d) computing an aggregated entropy hash; (e) computing an Entropy Confidence Score from a plurality of weighted dimensions including freshness, diversity, independence, manipulation resistance, verification success, and availability; (f) assigning a grade classification based on the computed score; (g) constructing a structured digital object containing the aggregated entropy, the aggregated hash, the individual source records, the confidence score, and metadata; and (h) outputting the structured digital object as a Verifiable Entropy Object.
2. A computer-implemented method for computing an Entropy Confidence Score, comprising: (a) computing a freshness score based on time elapsed since entropy generation; (b) computing a diversity score based on the number of unique independent entropy sources; (c) computing an independence score based on the number of unique entropy source type categories; (d) computing a manipulation resistance score based on the difficulty of controlling the aggregated output; (e) computing a verification success score based on the rate of successful source verification; (f) computing an availability score based on source availability at generation time; (g) applying a weighted sum of the computed dimension scores to produce a composite score; (h) applying penalty rules when one or more sources have fallen back to local

cryptographic randomness; and (i) assigning a grade classification based on defined score boundaries.

3. A computer-implemented method for signing a verifiable entropy object, comprising: (a) constructing a canonical signing payload by deep-sorting all object keys alphabetically, removing undefined values, preserving null values, excluding self-referential signature fields and post-anchor transaction fields, and applying stable JSON serialization; (b) computing a cryptographic hash of the canonical payload; (c) generating a digital signature over the hash using a provider private key; and (d) embedding the signature, provider public key, provider address, and canonical hash in the object's proof package.
4. A computer-implemented method for verifying a verifiable entropy object at multiple trust levels, comprising: (a) validating the object schema and required field presence; (b) recomputing the entropy hash and comparing it to the stored hash; (c) if a provider signature is present, reconstructing the canonical signing payload, computing its hash, recovering the signer address from the signature, and comparing it to the claimed provider address; (d) if a blockchain anchor is present, querying the on-chain smart contract to retrieve the stored root hash and comparing it to the anchor metadata in the object; (e) evaluating the object against an optional consumer policy specifying minimum confidence score, minimum source count, and anchoring requirements; and (f) assigning a verification level selected from the group consisting of: structurally valid unsigned, cryptographically verified, anchored verified, policy failed, and invalid.
5. A computer-implemented system for verifiable entropy generation, comprising: (a) a plurality of entropy source adapters, each configured to retrieve entropy from an independent external source and produce a

source record; (b) an aggregation module that deterministically combines entropy from the plurality of source adapters; (c) a scoring module that computes an Entropy Confidence Score across a plurality of weighted dimensions; (d) a signing module that produces a deterministic canonical representation of the entropy object and generates a cryptographic signature; (e) an anchoring module that writes the entropy hash to a blockchain smart contract and records the transaction metadata; and (f) a verification module that independently verifies the entropy hash, provider signature, and blockchain anchor at multiple trust levels.

6. The method of claim 1, further comprising classifying the verifiable entropy object into one of a plurality of object classes based on its trust characteristics, wherein the classes include: raw source, composite, anchored, and decision-linked.
7. The method of claim 2, wherein the penalty rules comprise reducing specific dimension scores by defined amounts based on the number of sources that have fallen back to local cryptographic randomness, and capping the total score and grade when all sources are in fallback.
8. The method of claim 3, wherein the canonical signing payload always sets the anchor field to null because the signature is computed before blockchain anchoring occurs, and the anchor transaction metadata is not known at signing time.
9. The method of claim 4, wherein the consumer policy system returns an explicit error when requirements cannot be satisfied, rather than returning a degraded entropy object at a higher trust level.
10. The method of claim 1, further comprising recording parent object identifiers in a lineage field, computing a lineage hash over the parent identifiers,

and enabling construction of a directed acyclic graph of entropy provenance.

11. The system of claim 5, wherein the plurality of entropy source adapters includes at least one public randomness beacon, at least one blockchain block hash source, and a fallback adapter that generates local cryptographic randomness with an associated quality penalty.
12. The method of claim 4, wherein the blockchain anchor verification comprises querying a smart contract function that returns the stored Merkle root for a given batch identifier, and comparing the returned root to the Merkle root recorded in the entropy object's anchor metadata.

ABSTRACT OF THE DISCLOSURE

A method and system for generating, scoring, signing, anchoring, and verifying Verifiable Entropy Objects. The system obtains entropy from multiple independent sources, aggregates it deterministically, computes a multi-dimensional Entropy Confidence Score that quantifies quality across freshness, diversity, independence, manipulation resistance, verification success, and availability dimensions, and packages the result as a structured digital object with source provenance records. A canonical signing protocol transforms the objects into deterministic payloads for cryptographic signing, enabling independent signature verification through address recovery. Optionally, the entropy hash is anchored to a blockchain smart contract for immutable timestamping. A multi-level verification framework distinguishes structural validity from cryptographic verification from blockchain-anchored verification from policy failure from invalidity. Consumer policies specify minimum quality requirements with honest failure when requirements cannot be met. The system transforms entropy from a disposable numerical output into a portable, auditable, and independently verifiable digital object.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 is a block diagram illustrating the overall system architecture for generating a Verifiable Entropy Object.

FIG. 2 is a diagram illustrating the computation of the Entropy Confidence Score from six weighted dimensions.

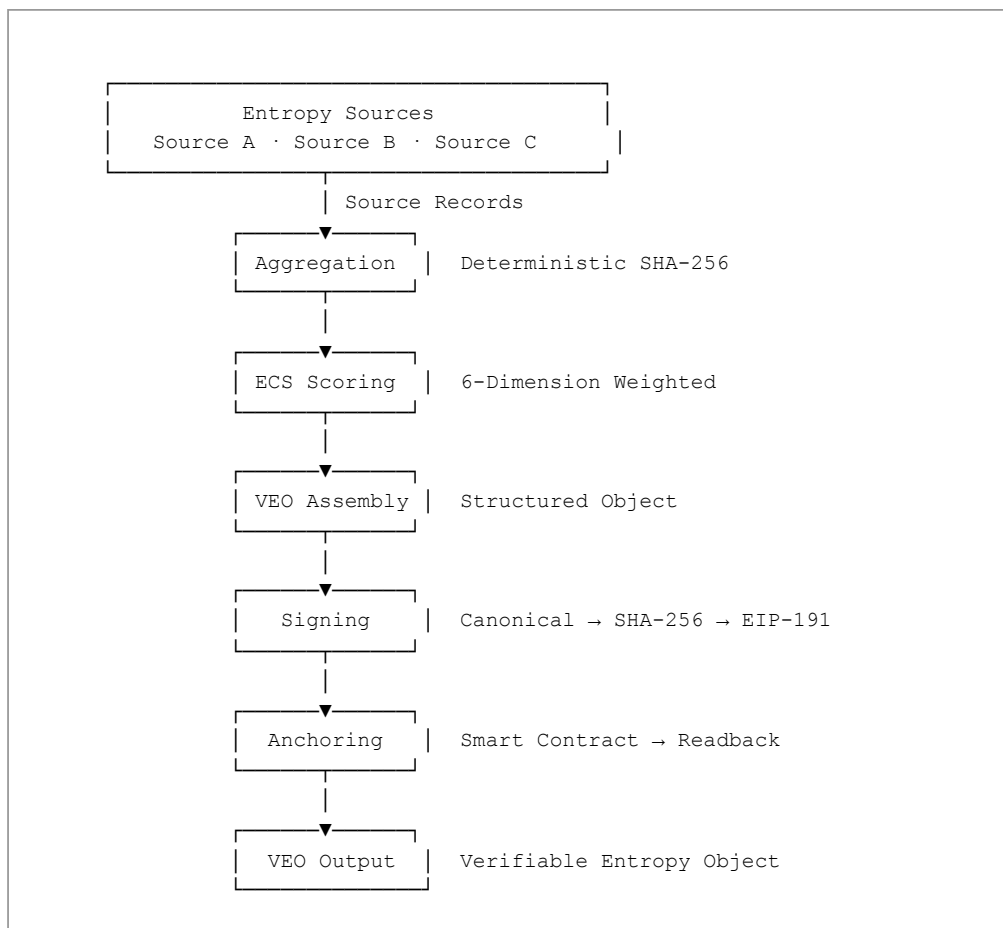
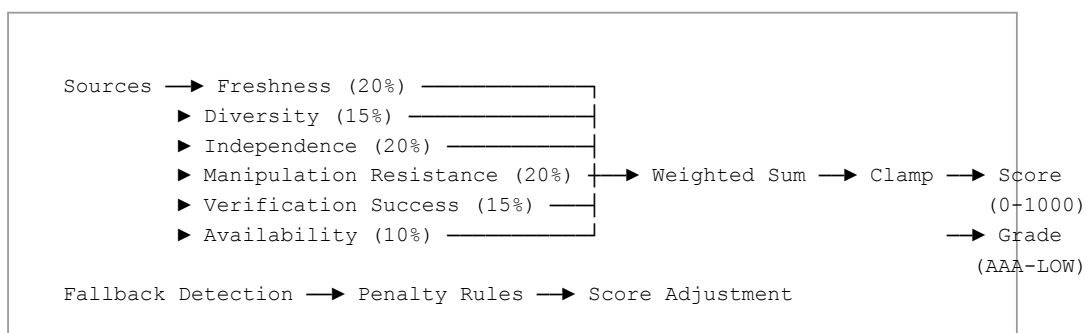
FIG. 3 is a flowchart illustrating the multi-level verification process.

FIG. 4 is a diagram illustrating the Entropy Provenance Graph showing object lineage.

FIG. 5 is a diagram illustrating the VEO object class hierarchy.

DRAWINGS

FIG. 1 — System Architecture

**FIG. 2 — Entropy Confidence Score Computation****FIG. 3 — Verification Flow**

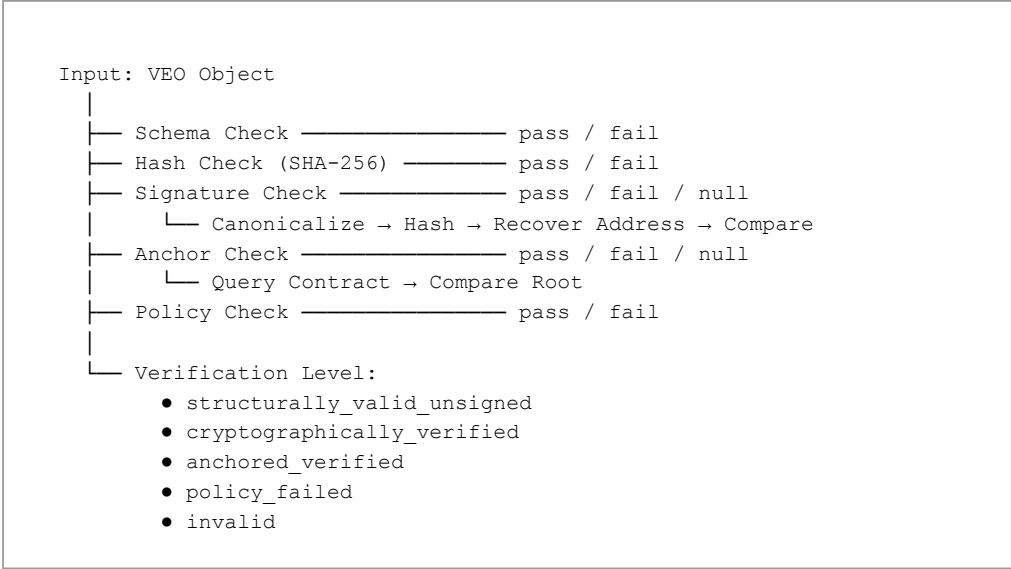


FIG. 4 — Entropy Provenance Graph

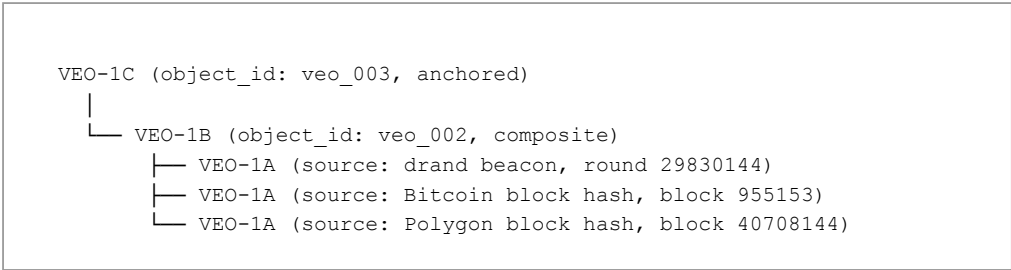
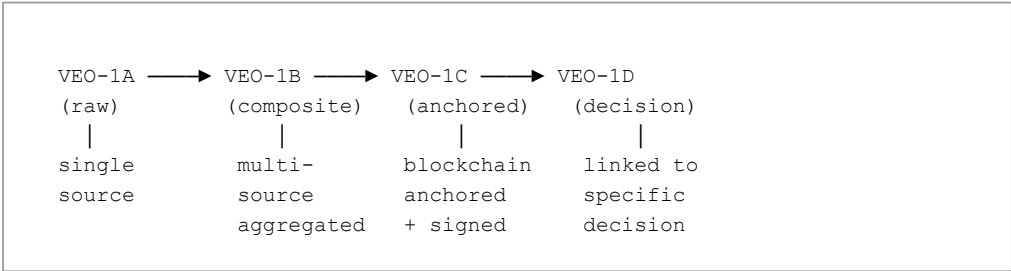


FIG. 5 — Object Class Hierarchy



VEO Patent Strengthening Instructions

Non-Provisional Upgrade — Patent #3 — v2

Method and System for Establishing Portable, Verifiable Trust
Over Non-Deterministic Computation

Date: June 25, 2026

Scope: Expand protection from a specific implementation into a foundational infrastructure layer.

Rule: Do NOT rewrite the existing invention. Preserve all existing content and ADD the following concepts, embodiments, and claim coverage.

★ The Core Reframe: Trust Over Uncertainty

This is the most important instruction in this document.

The invention is NOT entropy.

The invention is **trust over uncertainty**.

Those are different things.

Current Framing Throughout the Provisional

- Generate entropy
- Score entropy
- Verify entropy

Required Framing Throughout the Non-Provisional

- Establish trust in non-deterministic computation

The Analogy

Standard	Did NOT Invent	DID Invent
PDF	Documents	Portable trusted documents
JWT	Identity	Portable trusted identity
VEO	Randomness	Portable trusted uncertainty

Concrete Instructions

1. In the **Abstract**, replace language about "generating entropy" with language about "establishing verifiable trust over non-deterministic processes."
2. In every **independent claim**, ensure the language describes establishing trust rather than merely generating or scoring entropy. The entropy is the input. The trust assertion is the output.
3. Add a section titled "**Trust Establishment**" or "**Trust Over Uncertainty**" that explicitly describes the system's purpose as creating independently verifiable trust assertions about non-deterministic computations.
4. Frame **ECS** not as "entropy quality scoring" but as "trust quantification for non-deterministic outputs" — this broadens applicability beyond randomness into any system where outputs involve uncertainty.
5. Frame the **verification framework** not as "checking an entropy object" but as "progressively evaluating the trust level of an uncertainty artifact."

The Test

A reader should finish the non-provisional believing the invention covers any system where:

- An output involves uncertainty or non-determinism
- A structured object captures the uncertainty's provenance and quality
- Cryptographic and/or immutable proofs establish trust
- Independent verification is possible without trusting the generator

That scope includes AI decisions, autonomous systems, simulations, governance, gaming — and anything else where non-deterministic computation needs trust infrastructure.

Goal: Protect the category, not just the implementation.

1. Reframe VEO as a Universal Uncertainty Artifact

Current framing: Verifiable Entropy Object is a structured entropy object.

Expand to: A Verifiable Entropy Object (VEO) is a **portable, machine-verifiable uncertainty artifact** that encapsulates one or more entropy sources, provenance information, quality assessments, cryptographic attestations, trust metadata, and verification evidence.

Goal: Protect the concept of uncertainty becoming a transferable, auditable digital artifact.

2. Add AI Decision Attestation Embodiments

The provisional mentions AI in the background but does not claim it.

Add embodiments covering: LLM tool selection, agent routing, multi-agent arbitration, resource allocation, autonomous task assignment, sampling decisions, autonomous execution approval, model selection, AI governance workflows.

Example claim language: A computer-implemented method comprising: generating a verifiable entropy object; associating the verifiable entropy object with an autonomous agent decision; recording a decision outcome linked to the verifiable entropy object; enabling independent verification of the randomness influencing the decision.

Goal: Directly cover future AI infrastructure.

3. Introduce Decision Receipt Concepts

New embodiment: Decision Receipt / Decision Attestation Record.

```
{
  "decision_id": "...",
  "decision_type": "tool_selection",
  "selected_option": "Tool B",
  "confidence": 0.81,
  "veo_id": "...",
  "signature": "...",
```

```
"anchor": "..."  
}
```

Covers: Human decisions, AI decisions, multi-agent decisions, governance votes, automated execution events.

Goal: Create continuation opportunities around auditable decision systems.

4. Generalize Signature Systems

Current: Ethereum, EIP-191, secp256k1.

Expand to: secp256k1, Ed25519, RSA, post-quantum signatures, future asymmetric systems.

Language: "Any asymmetric cryptographic signature scheme capable of proving authenticity and permitting verification."

Goal: Prevent easy design-around.

5. Generalize Hash Functions

Current: SHA-256.

Expand to: SHA-256, SHA-3, Keccak, Blake2, Blake3, future cryptographic hash functions.

Language: "One or more cryptographic hash functions."

Goal: Future-proof claim scope.

6. Generalize Anchoring Targets

Current: Blockchain anchoring.

Expand to: Public blockchains, consortium chains, private chains, append-only ledgers, transparency logs, notarization networks, distributed timestamping systems.

Claim: "Immutable verification substrate" instead of only "blockchain."

Goal: Protect the trust model rather than a specific ledger.

7. Verification as a Trust State Machine

Current levels: Structurally Valid, Cryptographically Verified, Anchored Verified, Policy Failed, Invalid.

Describe as: A trust state machine where an object transitions between verification states.

Additional states: pending, partially verified, expired, revoked, superseded.

Claim: "Assigning an object to one of a plurality of trust states based on progressively evaluated verification criteria."

Goal: Separate patentable value from the entropy itself.

8. ECS Beyond Fixed Weights

Add embodiments: Dynamic weighting, policy-based weighting, industry-specific weighting, machine-learned weighting, regulator-defined weighting.

Language: "The dimensions and weighting coefficients may be modified based on application requirements."

Goal: Avoid limiting protection to current percentages.

9. Interoperability Claims

Embodiments: One provider generates, another verifies, another consumes.

Key statement: "A verifier need not trust the original generator."

Goal: Standards-layer positioning.

10. Entropy-as-a-Service

Services: Entropy generation, VEO issuance, VEO verification, ECS scoring, anchoring.

Claim: "Issuing verifiable entropy objects through a network-accessible service."

Goal: Protect future API businesses.

11. Multi-Party Governance

Examples: DAO voting, lottery systems, regulated gaming, compliance approvals, corporate governance.

Pattern: VEO referenced during outcome generation; outcome remains independently auditable.

Goal: Increase licensing opportunities.

12. AI Audit Trail

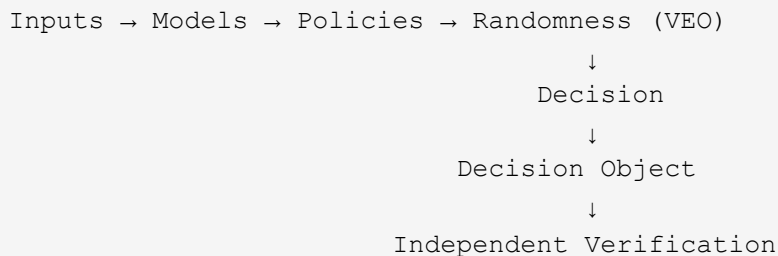
Covers: Decision provenance, replayability, regulatory review, model governance, post-event investigation.

Example: "A third party may reconstruct the decision context by verifying the associated VEO and related decision records."

Goal: Position VEO as future AI governance infrastructure.

14. Reserve Continuation: Patent #4 (VDO)

The non-provisional should explicitly leave room for a continuation covering **Verifiable Decision Objects (VDO)**.



VEO becomes one component of a larger decision-verification stack:

- VEO provides the entropy foundation
- VDO wraps the decision that used that entropy
- The VDO references the VEO, the model, the inputs, and the policy
- The entire chain is independently verifiable

Application domains for Patent #4: AI agents, autonomous vehicles, robotics, financial trading, healthcare workflows, enterprise automation.

To reserve this continuation: Ensure the non-provisional specification includes at least one embodiment describing a decision object that references a VEO, without fully specifying the VDO protocol. This creates the priority chain.

Strategic Summary

The non-provisional should no longer read as:

"A better random number system."

It should read as:

"A system for establishing portable, verifiable trust over non-deterministic computation — across software systems, AI agents, governance systems, gaming systems, and distributed networks."

Patent Family Progression

Patent	What It Protects	Category
#1	Generate verifiable randomness	Randomness generation
#2	Prevent randomness manipulation and preview	Randomness integrity
#3	Package into a portable, auditable trust artifact	Trust over uncertainty
#4 (future)	Use trust artifacts to verify AI and automated decisions	Decision verification

Each patent extends the previous one while opening a larger application domain. That progression maximizes long-term patent value, continuation opportunities, licensing potential, and acquisition attractiveness.

Reference Documents

#	Document	Purpose
1	Patent3_VEO_Provisional.pdf	Filed provisional application

2	RFC-0001-VEO1.md	Frozen protocol specification
3	ECS-v1.md	Frozen scoring specification
4	OpenRNG_Whitepaper_v1.md	Category-defining whitepaper
5	Agent Arbiter demo	Working AI decision attestation example